Informe de Ingeniería del Motor de Asignación Automática de Horarios de Clases

Documento 1 de la Entrega Final del Módulo 3. Construcción, Validación y Sustentación

Video de respaldo en YouTube [ENLACE DEL VIDEO]

Jesús David Beltrán Zárate

Juan Sebastián Albarracín Isaza

Daniel Felipe Delgadillo Sepúlveda

Martín Daniel Porto Gutiérrez

Programa de Ingeniería de Sistemas, Facultad de Ingeniería, Universidad El Bosque

Proyecto Núcleo 2 (15319), 2026-2, Grupo 5

Ing. Samuel Galindo Pinzón

5 de noviembre de 2026

Repositorio https://github.com/jesusBeltran79/ProgHorarioClase

Normas APA 7.ª edición

Introducción

Este informe cierra el proyecto del Grupo 5. El Módulo 1 definió el problema y el alcance. El Módulo 2 diseñó el motor y lo midió con un prototipo. En el Módulo 3 el equipo construyó el sistema completo con una API REST, una interfaz web, una base de datos PostgreSQL y seguridad con dos perfiles. Al final del módulo amplió el alcance con la gestión de datos desde la interfaz, las cuentas de docentes, la publicación del horario y el movimiento de sesiones arrastrándolas. Este documento presenta el diseño en su versión final, explica qué cambió y por qué, muestra con evidencia cómo responde el sistema construido y declara lo que no se logró.

La entrega tiene cuatro documentos que se leen juntos. Este es el Documento 1. Informe de Ingeniería (PN2_Grupo5_Doc1_Informe.pdf). Los diagramas UML finales y el modelo de datos están en el Documento 2. Anexos Técnicos (PN2_Grupo5_Doc2_Anexos.pdf) con la numeración de figuras A1 a E2. La arquitectura en detalle, el diccionario de datos, la instalación desde cero y la guía para mantener las restricciones están en el Documento 3. Manual Técnico (PN2_Grupo5_Doc3_ManualTecnico.pdf). La operación paso a paso para el administrador está en el Documento 4. Manual de Usuario (PN2_Grupo5_Doc4_ManualUsuario.pdf).

El código está en el repositorio del equipo. Las cifras que aparecen aquí salen de los archivos de la carpeta docs/evidencias, que se generan con las pruebas automáticas y la integración continua. Cada cifra cita el archivo del que sale.

Problema y Alcance

El problema no cambió desde el Módulo 1. El sistema debe asignar a cada sesión semanal de cada grupo una tupla formada por un docente, un aula y una franja horaria sin violar las restricciones duras del caso (Lopez-Cruz, s.f.). Las franjas válidas salen de una grilla que empieza a las 07:00 con bloques de dos horas, no cruza el intervalo de 12:00 a 13:00 y cierra el sábado a las 13:00. Con esos parámetros hay 32 franjas.

El alcance comprometido incluye ocho restricciones duras y dos reglas de validación de los datos de entrada. RD-01 y RD-02 impiden los choques de docente y de aula. RD-03 limita las sesiones a las franjas válidas. RD-04 respeta la disponibilidad y la habilitación del docente. RD-05 exige las características del aula y RD-06 la capacidad suficiente. RD-07 limita cada curso a entre una y cuatro sesiones de dos horas. RD-08 exige que toda sesión quede asignada para que el horario sea válido. RV-01 marca los grupos que superan el tope de 40 con su tolerancia y RV-02 marca para cierre los grupos con menos de 10 matriculados. Las restricciones blandas RB-01 a RB-04 y las exclusiones RE-01 a RE-04 quedaron fuera del MVP desde el Módulo 1 y siguen fuera.

Diseño Final y Cambios Justificados

El diseño del Módulo 2 se mantiene en lo esencial. El motor sigue siendo un backtracking con la sesión más restringida primero, las restricciones duras siguen el patrón Strategy con la interfaz RestriccionDura y las reglas de validación tienen su propio contrato ReglaValidacionEntrada. Lo que cambió vino de la construcción. Algunos cambios corrigen errores que el equipo encontró en su autodiagnóstico. Otros aparecieron al conectar el motor con la API y la base de datos. La Tabla 1 los reúne y las secciones siguientes explican los más importantes.

Tabla 1

Cambios del diseño entre el Módulo 2 y el sistema construido

Elemento	Diseño del Módulo 2	Sistema construido	Justificación
Seguridad	Componente de seguridad de Spring Boot sin diseño detallado	JWT firmado con JJWT 0.12.6, contraseña con bcrypt y reglas por perfil ADMIN y DOCENTE en las rutas /api	Cumple RNF-06 sin sesión de servidor. Era el Ajuste 3 del autodiagnóstico
Formato de intercambio	CSV o JSON	Solo CSV con el formato de la Tabla D2 de los Anexos del M2	JSON nunca tuvo especificación. Se declara fuera del alcance logrado
Actores y perfiles	Un solo actor con perfil ADMIN y datos que entraban solo por CSV	Perfiles ADMIN y DOCENTE, gestión de catálogos desde la interfaz y portal del docente	La coordinación necesita corregir un dato sin rehacer un CSV y cada docente necesita consultar su propio horario

Elemento	Diseño del Módulo 2	Sistema construido	Justificación
Capa de aplicación	ControladorHorarios llamaba al dominio y ensamblaba las reglas	ControladorHorarios delega en ServicioHorarios y las reglas se registran en ConfiguracionMotor	El controlador REST queda sin lógica y el ensamblaje está en un solo lugar
Umbral de RV-01	Math.floor que daba 38 con tope 35	Techo de CAP_MAX × (1 + TOL) que da 44 con tope 40 y 39 con tope 35	Corrige la discrepancia con el Módulo 1. Era el Ajuste 1 del autodiagnóstico
Acceso a datos	Entidades JPA	Repositorios con JdbcTemplate y esquema definido en scripts de Flyway	El esquema queda en un solo lugar y el núcleo no carga anotaciones de persistencia
Restricción RD-10	No existía	RestriccionDocenteUnicoGrupoRD10 exige el mismo docente en todas las sesiones de un grupo	La construcción mostró que el caso la daba por supuesta. Se agregó sin tocar el motor
Técnica de interacción	Selección asistida. El M2 descartó arrastrar y soltar porque el error solo se explica después de soltar	Arrastrar y soltar con Angular CDK sobre la misma selección asistida. Al tomar la sesión se resaltan las celdas válidas y el servidor valida al soltar	El arrastre resuelve la objeción del M2 porque muestra los destinos válidos antes de soltar y no se salta ninguna regla
Orden de las franjas	Orden de la grilla	OrdenFranjas.intercalarDias prueba primero la misma hora en días distintos	Reparte las sesiones de un grupo en la semana sin agregar una regla nueva
Modelo de datos	19 tablas con la disponibilidad y la asignación guardadas por índice de franja	19 tablas. disponibilidad_docente y asignacion guardan día y hora de inicio	Un cambio de la grilla ya no deja datos que apunten a franjas inexistentes
Conteo inicial	Precálculo por franja como decisión pendiente	Conteo completo de tuplas por sesión	No se implementó en esta versión. Se declara en el Acta de Cierre

Nota. Los nombres de clases y archivos son los del código del repositorio. RD = restricción dura. RV = regla de validación.

Seguridad con JWT

El Módulo 2 nombraba un componente de seguridad de Spring Boot pero no lo diseñaba. El autodiagnóstico reconoció que el equipo no sabía defenderlo. En la construcción se eligió un esquema sin sesión de servidor. El administrador envía usuario y contraseña a POST /api/auth/login. ControladorAutenticacion compara la contraseña con el hash bcrypt de la tabla usuario y ServicioJwt entrega un token firmado. FiltroJwt revisa ese token en cada petición. El token lleva el rol de la cuenta y el docente al que está ligada. ConfiguracionSeguridad exige el rol ADMIN en /api/admin, /api/horarios y /api/inventarios y el rol DOCENTE en /api/mi. Un docente que intenta generar un horario recibe 403. Un token tiene vigencia de 480 minutos por

defecto. Las pruebas de SeguridadTest verifican que generar sin token responde 401, que una contraseña errada no entrega token y que un token alterado no da acceso.

Servicio de Aplicación entre la API y el Dominio

En el diseño del Módulo 2 ControladorHorarios recibía la petición, armaba la lista de reglas y llamaba al motor. Al construir la API ese controlador tenía que leer la base, reconstruir el problema en memoria, validar, generar, guardar y armar la respuesta. Eso mezclaba la capa web con la lógica. Se agregó ServicioHorarios para que haga ese trabajo y el controlador quedó con una línea por ruta. El ensamblaje de las reglas pasó a ConfiguracionMotor, que publica la lista de restricciones duras y el validador como componentes de Spring. El motor sigue recibiendo la lista por constructor y no conoce las clases concretas. La Figura A1 del Documento 2. Anexos Técnicos (PN2_Grupo5_Doc2_Anexos.pdf) muestra la estructura final.

Umbral de RV-01 con Techo

El Módulo 1 define el umbral de RV-01 como el techo de CAP_MAX × (1 + TOL). El prototipo del Módulo 2 usaba el piso y con tope 35 publicó 38 en lugar de 39. Con tope 40 los dos redondeos dan 44 y por eso el error no se veía. El método calcularUmbralMaximoGrupo de ConfiguracionParametros ahora usa el techo y resta un margen muy pequeño para que 40 por 1,10 no termine en 45 por el error del punto flotante. La prueba umbralConTecho de ConfiguracionParametrosTest confirma 44 con tope 40 y 39 con tope 35. La prueba tope35 de ParametrosApiTest confirma lo mismo a través de la API.

Restricción RD-10

Con solo RD-01 a RD-08 nada impide que las sesiones de un mismo grupo queden con docentes distintos. El caso de estudio da por supuesto que un grupo tiene un solo docente, pero el Módulo 1 no lo escribió como restricción y la construcción lo hizo visible. Se agregó la clase RestriccionDocenteUnicoGrupoRD10 y se registró en ConfiguracionMotor. No fue necesario modificar MotorAsignacion. Es la segunda vez que el diseño admite una regla nueva sin tocar el

motor y por eso se cita también como evidencia de modificabilidad. La prueba docenteUnico de RestriccionesDurasTest la verifica.

Módulo de Usuarios por Perfil

El Módulo 2 tenía un solo actor y los datos entraban solo por archivos CSV. Al revisar el sistema construido el equipo vio que eso no servía en la práctica. Para cambiar la disponibilidad de un docente había que editar un archivo y volver a cargar todo. Los docentes tampoco tenían forma de ver su horario. Por eso se agregaron cinco requisitos nuevos que se diseñaron en PLAN_USUARIOS_M3.md antes de construirlos. Son requisitos del Módulo 3 y no vienen del Módulo 1. Por eso en la matriz de trazabilidad su origen aparece como nuevo y su cadena empieza en el requisito.

RF-10 permite al administrador crear, editar y borrar docentes, aulas, materias y grupos desde la interfaz. ServicioCatalogos valida con las mismas reglas del cargador CSV, es decir RD-03 para las franjas, RD-07 para las sesiones, las características de aula y las referencias. Cualquier cambio borra los horarios generados porque dejan de corresponder a los datos. Los docentes habilitados de una materia se asignan arrastrando fichas entre dos listas con Angular CDK. RF-11 permite crear cuentas ADMIN o DOCENTE. Cada cuenta DOCENTE queda ligada a un docente y un docente tiene a lo sumo una cuenta. El sistema no deja borrar ni desactivar al último ADMIN activo ni a la cuenta con la que se inició sesión.

RF-12 agrega la publicación. Solo un horario COMPLETO se puede publicar y solo uno queda publicado a la vez. RF-13 permite mover una sesión arrastrándola a otra celda de la grilla. Al tomar la sesión la pantalla pide al servidor las alternativas y resalta las celdas válidas. Al soltarla se aplica la alternativa de esa franja y si no hay ninguna el servidor rechaza el movimiento con el código de la restricción que se violaría. Así el arrastre no se salta ninguna regla porque la última palabra la tiene el motor. Este cambio contradice una decisión explícita del Módulo 2, que eligió la selección asistida en lugar de arrastrar y soltar. La razón de aquella decisión era que al arrastrar el usuario puede soltar en una celda inválida y el sistema solo

explica el error después. El arrastre construido evita ese problema porque usa las mismas alternativas de la selección asistida para marcar las celdas válidas antes de soltar. La selección asistida y la asignación manual siguen disponibles en el panel lateral. RF-14 da al docente un portal con su horario publicado, su disponibilidad semanal y el cambio de su contraseña. Cuando el docente cambia su disponibilidad el horario publicado no se toca y el cambio se usa en la siguiente generación.

La migración V3__usuarios_y_publicacion.sql amplía la tabla usuario con el rol DOCENTE, la columna id_docente y la columna activo, y agrega la columna publicado a ejecucion_horario. El núcleo del motor no cambió. Las pruebas CatalogosApiTest y CuentasApiTest y el recorrido e2e/usuarios.mjs verifican el módulo. Las capturas de ese recorrido están en el Documento 4. Manual de Usuario (PN2_Grupo5_Doc4_ManualUsuario.pdf).

Hallazgos del Autodiagnóstico

El autodiagnóstico del Módulo 2 dejó tres ajustes con fecha límite del 5 de noviembre. El Ajuste 1 pedía corregir el umbral de RV-01 y quedó resuelto como se explicó arriba. El Ajuste 2 pedía que CargadorInventariosCSV, ExportadorHorarioCSV y ControladorHorarios existieran en el código y que se decidiera el formato JSON. Las tres clases existen y las prueban CargadorInventariosCSVTest, MotorAsignacionTest y FlujoHorarioTest. JSON queda fuera del alcance y solo se intercambia CSV. El Ajuste 3 pedía diseñar la seguridad del administrador con JWT y bcrypt y quedó resuelto con las clases del paquete seguridad. El autodiagnóstico también señaló que la comparación de versiones de RD-09 no aparecía en los anexos. Esa comparación es la medida del escenario de modificabilidad de este informe.

Arquitectura Final

El sistema es un monolito modular en cuatro capas que corre en tres contenedores. La interfaz web en Angular 18.2 la sirve nginx 1.27 en el puerto 4200 y redirige las rutas /api al backend. El backend en Java 21 con Spring Boot 3.3.5 corre en el puerto 8080. PostgreSQL 16

guarda los datos en el puerto 5432. Todo se levanta con docker compose up --build. La Figura E1 del Documento 2. Anexos Técnicos (PN2_Grupo5_Doc2_Anexos.pdf) muestra los componentes y la Figura E2 el despliegue.

La decisión que sostiene la arquitectura se mantuvo. Los paquetes dominio, configuracion, restricciones, validacion y motor no importan Spring. El motor trabaja solo en memoria. ServicioHorarios lee los inventarios y los parámetros de la base una vez antes de generar y guarda el resultado una vez al terminar. Dentro del ciclo del backtracking no hay acceso a disco ni a red. Por eso el núcleo es el mismo que se midió en el Módulo 2 y sus pruebas corren sin levantar la aplicación.

La base de datos protege dos restricciones por su cuenta. La tabla asignacion tiene una restricción UNIQUE sobre ejecución, docente, día y hora y otra sobre ejecución, aula, día y hora. La prueba uniqueEnAsignacion de RestriccionesBaseDatosTest inserta a mano un choque de docente y uno de aula y comprueba que PostgreSQL los rechaza. RD-01 y RD-02 quedan así protegidas en el motor y en la base.

El algoritmo no cambió en su estructura. Cuenta para cada sesión las tuplas que pasan las restricciones locales, ordena las sesiones de menor a mayor dominio, recorre con backtracking evaluando primero las reglas locales y, si no encuentra solución, diagnostica cada sesión pendiente con el número de tuplas que bloquea cada código. El Documento 3. Manual Técnico (PN2_Grupo5_Doc3_ManualTecnico.pdf) lo describe en detalle y la Figura B1 del Documento 2. Anexos Técnicos (PN2_Grupo5_Doc2_Anexos.pdf) lo muestra como diagrama de actividades.

Análisis Final de Atributos de Calidad

El Módulo 2 eligió la arquitectura por dos atributos que se oponen. El desempeño del motor (RNF-01 y RNF-02) pide pocas capas dentro del ciclo de búsqueda. La facilidad para cambiar reglas (RNF-03) pide separar cada regla en su clase. Con el sistema construido el equipo puede decir cómo quedó cada uno.

Desempeño y Escalabilidad

RNF-01 pide generar el horario del conjunto de sustentación en 60 segundos o menos. RNF-02 pide procesar el triple de datos sin cambiar la estructura. Las mediciones del Módulo 2 ya mostraban mucho margen en 1x, 2x y 3x. En el Módulo 3 se repitieron sobre el código final y se extendieron a 5x y 10x con la prueba de estrés. El escenario de escalabilidad de la sección siguiente presenta los resultados.

Con el código final RNF-01 se cumple con un margen de cuatro órdenes de magnitud. El conjunto de sustentación de 1x se generó en 6 ms en el peor caso frente a un límite de 60.000 ms. RNF-02 también se cumple porque las escalas 3x, 5x y 10x corrieron con las mismas clases y la misma estructura de datos y todas terminaron con el horario completo. El tiempo crece más rápido que los datos. Pasar de 1x a 10x multiplica las sesiones por 10 y el tiempo promedio por unos 280 en el perfil holgado. Aun así la peor corrida de 10x tardó 1,24 s. La Sección de escenarios presenta las cifras completas.

El primer cuello de botella identificado en el Módulo 2 fue el conteo inicial, que crece con $N \times |D| \times |A| \times F$. Ese conteo sigue completo en el código final. Si el tiempo en 10x lo muestra como problema, la mejora identificada es precalcular por franja y su costo pasa a $N \times F \times (|D| + |A|)$.

Modificabilidad

RNF-03 pide agregar una regla sin modificar el motor. La construcción lo probó dos veces. La primera fue RD-09, la regla de demostración que prohíbe dos sesiones del mismo grupo el mismo día. La segunda fue RD-10, que entró al sistema de verdad. En ambos casos el cambio fue una clase nueva y una línea de registro. Los parámetros del semestre también se cambian sin recompilar desde la pantalla de parámetros o con PUT /api/parametros.

Explicabilidad y Seguridad

RNF-04 pide que cada conflicto diga qué regla lo causó. Cada RestriccionDura trae su código y el motor cuenta cuántas tuplas bloquea cada uno. La API devuelve esas causas en el

campo conflictos del Resultado y la reubicación rechazada devuelve los códigos RD que violaría. RNF-06 pide acceso restringido y trazabilidad de quién generó cada horario. Lo cubren el JWT con el rol de cada cuenta, las reglas por ruta de ConfiguracionSeguridad y la columna id_usuario de ejecucion_horario.

Usabilidad

RNF-05 pide que una persona no técnica complete el ciclo en 10 minutos o menos con el manual. La interfaz del administrador conserva las tres pantallas del Módulo 2 para el ciclo de generación y agrega cuatro de gestión. El docente tiene un portal aparte con tres pantallas. El ajuste del horario combina la selección asistida del Módulo 2 con arrastrar y soltar.

La interfaz construida se recorrió de punta a punta con un guion automático en Playwright (e2e/capturas.mjs). El guion inicia sesión, carga los datos, revisa los parámetros, genera, reubica una sesión, exporta y fuerza las rupturas, y terminó sin errores (docs/evidencias/capturas/recorrido.log). Ese recorrido demuestra que el ciclo funciona pero no mide el tiempo de una persona no técnica.

Además del recorrido automático, Martín Daniel Porto Gutiérrez realizó la prueba con usuarios de la interfaz construida que el Módulo 2 dejó pendiente en su Tabla 5. Con esa prueba se cierra el pendiente de usabilidad.

Escenarios de Atributos de Calidad

Los escenarios siguen el formato de estímulo, contexto, respuesta y medida de respuesta (Bass et al., 2021). Cada medida se tomó sobre el sistema construido y cita el archivo de evidencia.

Escenario de Escalabilidad

Tabla 2
Escenario de escalabilidad

Parte	Descripción
Fuente	Coordinador académico con perfil administrador

Parte	Descripción
Estímulo	Genera el horario con 1x, 3x, 5x y 10x del conjunto de sustentación de 10 docentes, 5 aulas y 20 asignaturas
Artefacto	MotorAsignacion con EstadoHorario indexado y orden de la sesión más restringida primero
Contexto	Operación normal con los perfiles de datos holgado y exigente del Módulo 2
Respuesta	El motor entrega el horario completo o el reporte de conflictos sin cambiar código ni estructura de datos
Medida	Tiempo de 60 s o menos en 1x (RNF-01), sin cambios de estructura hasta 3x (RNF-02) y tiempo y nodos reportados hasta 10x

Tabla 3

Tiempo y nodos explorados por escala y perfil de datos

Perfil	Escala	Sesiones	Promedio (ms)	Mediana (ms)	Peor caso (ms)	Nodos promedio	Completos
Holgado	1x	40	3,8	3	6	9.882	5 de 5
Holgado	3x	120	89,4	75	142	231.936	5 de 5
Holgado	5x	200	143,0	139	155	1.093.954	5 de 5
Holgado	10x	400	1.070,0	1.062	1.105	8.393.044	5 de 5
Exigente	1x	40	1,2	1	2	18.078	5 de 5
Exigente	3x	120	35,4	35	37	422.948	5 de 5
Exigente	5x	200	159,4	160	164	1.950.337	5 de 5
Exigente	10x	400	1.201,8	1.188	1.238	14.005.383	5 de 5

Nota. Cinco semillas por celda con un límite de 60.000 ms por corrida. Datos de docs/evidencias/estres/resumen.txt y estres.csv. Medido en GitHub Actions con 4 núcleos Intel Xeon Platinum 8370C a 2,80 GHz, 15 GB de memoria y OpenJDK Temurin 21.0.13 (docs/evidencias/entorno.txt).

La respuesta cumple las tres medidas del escenario. En 1x la peor corrida fue de 6 ms, muy lejos de los 60 s de RNF-01. Hasta 3x y más allá no hubo que cambiar código ni estructura. Las 40 corridas terminaron con el horario completo y ninguna llegó al límite de tiempo. El dato que más crece es el número de nodos, que en 10x exigente pasa de 14 millones. Eso confirma lo que anticipó el Módulo 2. El backtracking y el conteo inicial crecen más rápido que los datos y serán el primer límite si la universidad crece mucho más allá de 10x. El perfil exigente exploró más nodos que el holgado en todas las escalas. Aun así fue más rápido en 1x y 3x y solo tardó

más en 5x y 10x. El equipo no aisló la causa de esa diferencia y la deja registrada como observación.

Escenario de Modificabilidad

Tabla 4
Escenario de modificabilidad

Parte	Descripción
Fuente	Desarrollador del equipo de mantenimiento
Estímulo	Agrega una restricción dura nueva (RD-09) y cambia el tope de matriculados de 40 a 35
Artefacto	Paquete restricciones, ConfiguracionMotor y ConfiguracionParametros
Contexto	Tiempo de desarrollo para la regla y tiempo de operación para el tope
Respuesta	La regla se agrega creando una clase y registrándola. El tope se cambia como parámetro
Medida	MotorAsignacion con cero líneas cambiadas en el diff de RD-09. Umbral de RV-01 igual a 39 sin recompilar

La medida se tomó con el diff completo que produce activar RD-09 sobre el código final (docs/evidencias/modificabilidad/rd09.diff). El resumen del diff toca dos archivos y MotorAsignacion.java no aparece porque no cambia ninguna línea.

```
.../horarios/aplicacion/ConfiguracionMotor.java    |  4 +++-
.../restricciones/RestriccionMismoDiaRD09.java     | 25 ++++++++++++++++++++++
2 files changed, 28 insertions(+), 1 deletion(-)
```

Con RD-09 activa el motor generó el horario completo de 42 sesiones y ningún grupo quedó con dos sesiones el mismo día (docs/evidencias/modificabilidad/rd09.txt). El mismo archivo registra que RD-09 no rechazó ninguna tupla. La razón es que el orden intercalado de franjas ya reparte las sesiones de cada grupo en días distintos antes de que la regla tenga que actuar. La medida del escenario no depende de eso porque mide cuánto código cambia. Para ver la regla trabajando hay una prueba aparte. ModificabilidadTest.nuevaRestriccion la ejecuta y verifica que ningún grupo queda con dos sesiones el mismo día.

El cambio de tope se hizo a través de la API con el sistema en marcha. La salida de docs/evidencias/modificabilidad/rv01_tope35.txt lo registra así.

```
API PUT /api/parametros capMax=35 respondió umbralMaximoGrupo=39
Umbral RV-01 con tope 40 = 44. Con tope 35 = 39 (techo de 35 x 1,10). Sin recompilar
```

RD-10 es una segunda evidencia del mismo atributo. Entró al sistema durante la construcción como una clase nueva y una línea en ConfiguracionMotor, y tampoco modificó MotorAsignacion.

Informe de Calidad y Pruebas

Las pruebas automáticas están en backend/src/test y corren con mvn verify en cada envío al repositorio mediante GitHub Actions. Se agrupan en dos paquetes. Las del paquete nucleo prueban el motor y las reglas sin Spring. Las del paquete api levantan la aplicación con H2 en modo PostgreSQL y prueban el sistema a través de la API con un token de administrador.

Pruebas Unitarias y de Integración

La Tabla 5 resume la última ejecución. Las 42 pruebas pasaron sin fallas ni errores en 10,84 s.

Tabla 5

Resultado de las pruebas automáticas por clase

Clase de prueba	Pruebas	Fallas	Qué verifica
api.CatalogosApiTest	2	0	Gestión de catálogos, errores RD-03, RD-07, REFERENCIA y DUPLICADO y generación con datos creados a mano
api.CuentasApiTest	3	0	Cuenta DOCENTE, acceso 403, publicación, portal, último ADMIN y horario parcial no publicable
api.FlujoHorarioTest	7	0	Ciclo completo por la API y rupturas de RD-01, RD-02, RD-06, RD-08 y carga
api.ParametrosApiTest	3	0	Tope 35 con umbral 39, cierre a las 20:00 y parámetro inválido
api.RestriccionesBaseDatosTest	1	0	Restricciones UNIQUE de asignacion
api.SeguridadTest	4	0	Token válido, sin token, clave errada y token alterado
nucleo.CargadorInventariosCSVTest	2	0	Carga de la demostración y rechazo fila por fila
nucleo.ConfiguracionParametrosTest	4	0	Grilla de 32 franjas, bloqueo del mediodía, cierre y umbral con techo
nucleo.EstresTest	3	0	Generación completa en tres escalas

Clase de prueba	Pruebas	Fallas	Qué verifica
nucleo.ModificabilidadTest	2	0	RD-09 sin tocar el motor y cambio de tope
nucleo.MotorAsignacionTest	3	0	Demostración completa, caso sobrerrestringido y exportación
nucleo.ReglasValidacionTest	2	0	RV-01 y RV-02
nucleo.RestriccionesDurasTest	6	0	RD-01, RD-02, RD-04, RD-05, RD-06 y RD-10
Total	42	0	

Nota. Fuente docs/evidencias/qa/pruebas_unitarias.txt. JUnit 5 con Surefire. Las mismas pruebas pasan en GitHub Actions en cada cambio. Ninguna prueba omitida.

Además de las pruebas del equipo, MotorAsignacionTest pasa cada horario generado por un verificador independiente que recorre las asignaciones y busca violaciones sin usar las clases de restricciones. La demostración salió COMPLETA con 42 de 42 sesiones en 2 ms y 11.706 nodos y el verificador no encontró violaciones (docs/evidencias/qa/motor.log).

Pruebas de Restricciones y de Ruptura

La guía pide demostrar que el sistema bloquea los choques de horario, aula y docente. El equipo preparó pruebas de ruptura que fuerzan cada error y comprueban la respuesta. La Tabla 6 resume qué fuerza cada una y qué debe responder el sistema.

Tabla 6

Pruebas de ruptura y respuesta esperada

Prueba	Qué fuerza	Respuesta del sistema
rupturaChoqueDocente (FlujoHorarioTest)	Mover una sesión a una franja donde su docente ya dicta	409 RESTRICCION_VIOLADA con RD-01
rupturaChoqueAula (FlujoHorarioTest)	Mover una sesión a un aula ocupada en esa franja	409 RESTRICCION_VIOLADA con RD-02
rupturaCapacidad (FlujoHorarioTest)	Mover un grupo grande a un aula pequeña	409 RESTRICCION_VIOLADA con RD-06
rupturaParcial (FlujoHorarioTest)	Generar el caso sobrerrestringido del Dr. Gómez	Horario PARCIAL con causas por código y exportación bloqueada con 409 HORARIO_PARCIAL
rupturaCarga (FlujoHorarioTest)	Cargar archivos con filas inválidas	Cada fila rechazada con archivo, fila, código y motivo

Prueba	Qué fuerza	Respuesta del sistema
uniqueEnAsignacion (RestriccionesBaseDatosTest)	Insertar a mano un choque de docente y uno de aula en la base	PostgreSQL rechaza las dos filas por las restricciones UNIQUE

Nota. Los datos de ruptura están en datos/ruptura. Los choques directos se fuerzan sobre los datos de datos/demo.

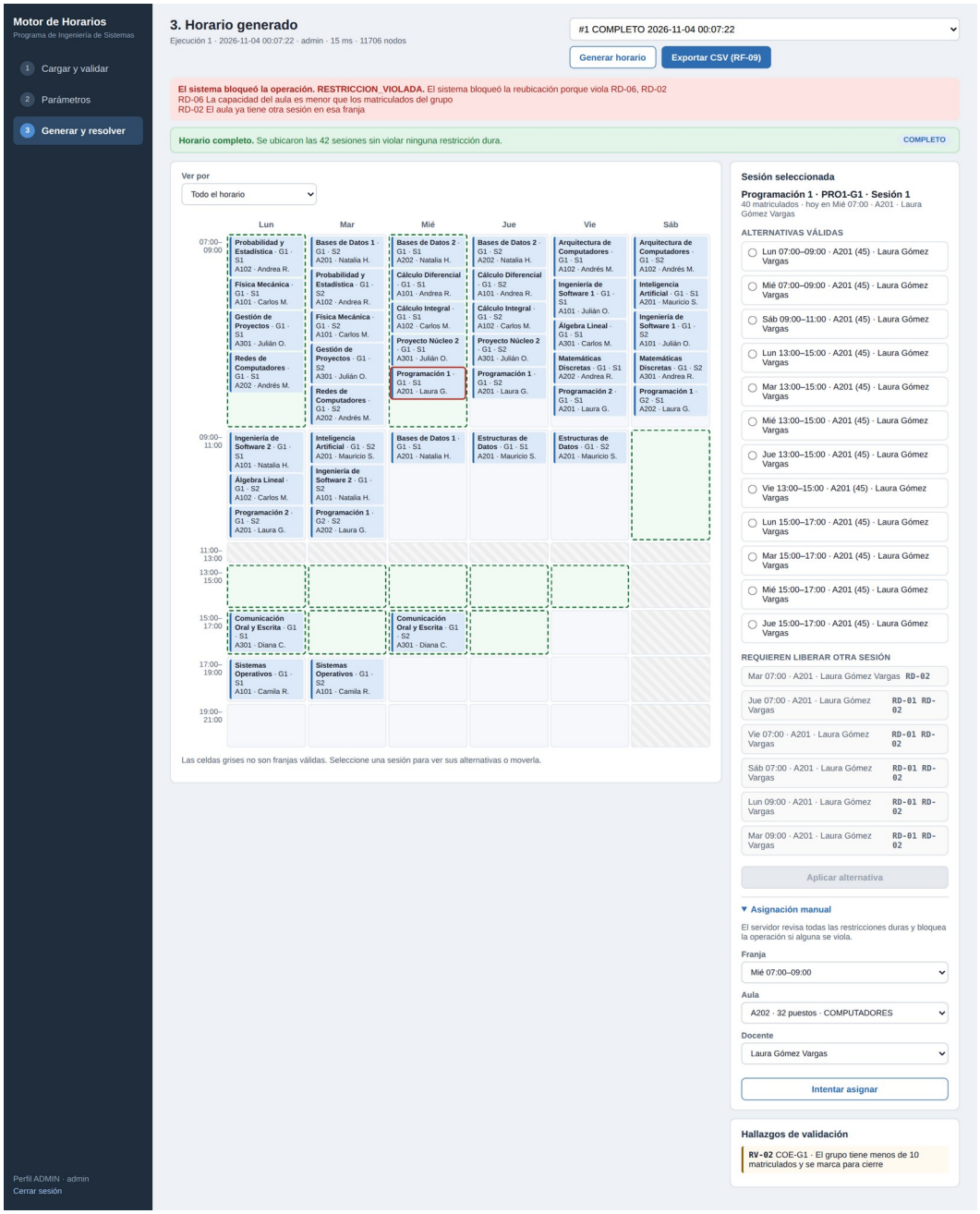
Las mismas rupturas se repitieron contra el sistema desplegado con Docker Compose y PostgreSQL 16, no solo contra la base de pruebas H2. El registro docs/evidencias/qa/ruptura_api_postgresql.log termina con cero fallas. Este es un extracto.

```
OK  Ruptura RD-06 mover PRO1-G1 de 40 al aula A202 de 32 -> HTTP 409
    ["RD-06 La capacidad del aula es menor que los matriculados del grupo",
     "RD-02 El aula ya tiene otra sesión en esa franja"]
OK  Ruptura RD-01 ARQ-G1-S1 a la franja de RED-G1-S1 -> HTTP 409
    ["RD-01 El docente ya tiene otra sesión en esa franja",
     "RD-02 El aula ya tiene otra sesión en esa franja"]
OK  Ruptura RD-02 ARQ-G1-S1 al aula A101 -> HTTP 409
    ["RD-02 El aula ya tiene otra sesión en esa franja"]
OK  RD-08 caso sobrerrestringido queda PARCIAL -> causas RD-04, RD-10, RD-06, RD-01, RD-
02
OK  Ruptura RD-08 exportar un horario PARCIAL -> HTTP 409 HORARIO_PARCIAL
OK  PostgreSQL rechazó el choque de docente RD-01 (D1 MARTES 07:00)
    duplicate key value violates unique constraint "uq_asignacion_docente_franja"
OK  PostgreSQL rechazó el choque de aula RD-02 (A204 MARTES 09:00)
    duplicate key value violates unique constraint "uq_asignacion_aula_franja"
Fin. Fallas 0
```

Las rupturas muestran que el sistema bloquea el error y además explica la causa con su código. Cuando una reubicación viola varias reglas la respuesta las lista todas. En el caso sobrerrestringido del Dr. Gómez la sesión CAL2-G2-S2 queda pendiente y el motor cuenta cuántas tuplas bloquea cada causa. RD-04 bloquea 183, RD-10 bloquea 96, RD-06 bloquea 64, RD-01 bloquea 9 y RD-02 bloquea 6 (docs/evidencias/qa/ruptura.log). RD-10 aparece como causa nueva frente al Módulo 2 porque la sesión 1 del grupo ya la dicta el Dr. Gómez.

Figura 1

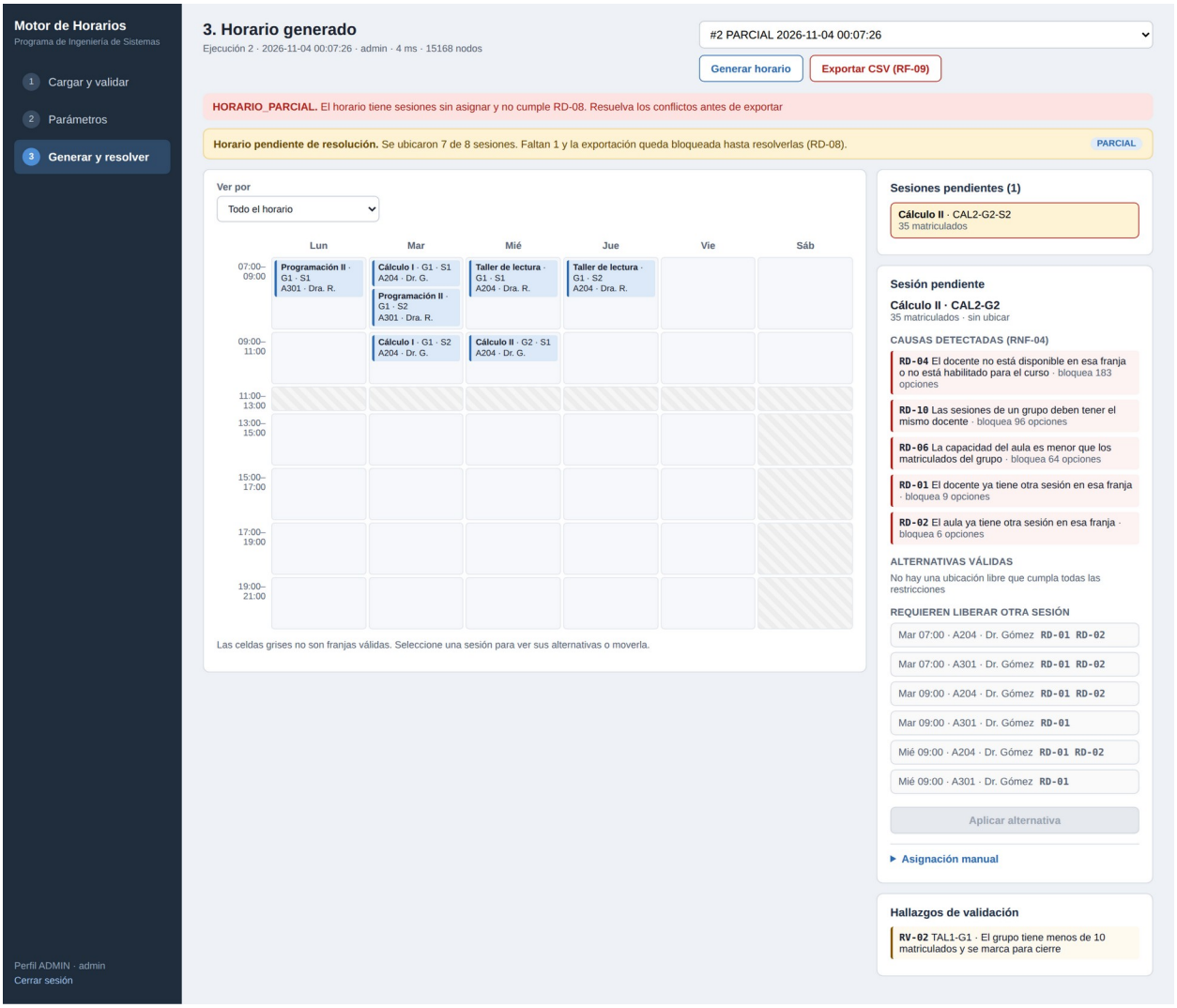
Ruptura de capacidad bloqueada en la interfaz



Nota. El administrador intenta asignar a mano un grupo de 40 estudiantes a un aula de 32 puestos. El sistema rechaza la operación y muestra RD-06 y RD-02.

Figura 2

Exportación bloqueada de un horario parcial



Nota. El caso sobrerrestringido queda con 7 de 8 sesiones. La exportación responde HORARIO_PARCIAL porque el horario no cumple RD-08.

Pruebas de Estrés

La prueba de estrés genera conjuntos de 1x, 3x, 5x y 10x con semillas fijas y ejecuta el motor con el mismo código del sistema. La corre el programa PruebaEstres en la integración continua y la prueba EstresTest en JUnit.

La Tabla 3 presenta los resultados. Con carga masiva de 10x, que son 100 docentes, 50 aulas y 400 sesiones, el motor generó el horario completo en las 10 corridas con un peor caso de

1,24 s. EstresTest repite tres escalas en cada ejecución de JUnit para que una regresión de desempeño rompa la integración continua.

Matriz de Trazabilidad Actualizada

La Tabla 7 completa la cadena que la guía exige verificar. Cada condición del Módulo 1 va a su requisito, al componente del diseño del Módulo 2, a la clase o archivo del código y a la prueba que la comprueba. Las pruebas se nombran por su clase y su método en backend/src/test.

Tabla 7

Matriz de trazabilidad de requisito a prueba

ID	Condición del M1	Requisito	Componente M2	Código	Prueba
RD-01	Choque de docente	RF-06, RF-07, RNF-04	RestriccionChoqueDocente RD01 y EstadoHorario	restricciones/ RestriccionChoqueDocenteRD01.java y UNIQUE uq_asignacion_docente_franja	RestriccionesDurasTest.choqueDocente, FlujoHorarioTest.rupturaChoqueDocente, RestriccionesBaseDatosTest.uniqueEnAsignacion
RD-02	Choque de aula	RF-06, RF-07, RNF-04	RestriccionChoqueAulaRD02 y EstadoHorario	restricciones/ RestriccionChoqueAulaRD02.java y UNIQUE uq_asignacion_aula_franja	RestriccionesDurasTest.choqueAula, FlujoHorarioTest.rupturaChoqueAula, RestriccionesBaseDatosTest.uniqueEnAsignacion
RD-03	Solo franjas válidas	RF-06, RF-08	ConfiguracionParametros y FranjaHoraria	ConfiguracionParametros.generarFranjasValidas y validación de idFranja en ServicioHorarios.reubicar	ConfiguracionParametrosTest.grillaPorDefecto y sinCruzarBloqueo
RD-04	Disponibilidad y habilitación	RF-01, RF-06, RNF-04	RestriccionDisponibilidadDocenteRD04	restricciones/ RestriccionDisponibilidadDocenteRD04.java	RestriccionesDurasTest.disponibilidad
RD-05	Características del aula	RF-02, RF-03, RF-06	RestriccionIdoneidadAulaRD05	restricciones/ RestriccionIdoneidadAulaRD05.java	RestriccionesDurasTest.idoneidad
RD-06	Capacidad del aula	RF-02, RF-04, RF-06	RestriccionCapacidadAulaRD06	restricciones/ RestriccionCapacidadAulaRD06.java	RestriccionesDurasTest.capacidad, FlujoHorarioTest.rupturaCapacidad
RD-07	De 1 a 4 sesiones	RF-03, RF-08	CargadorInventariosCSV, Curso y Sesion	CargadorInventariosCSV.cargarCursos y CHECK ck_curso_sesiones	CargadorInventariosCSVTest.cargaConErrores

ID	Condición del M1	Requisito	Componente M2	Código	Prueba
RD-08	Toda sesión asignada	RF-06, RF-07	MotorAsignacion, ResultadoAsignacion y ReporteConflicto	MotorAsignacion.generarHorario y ServicioHorarios.exportar	MotorAsignacionTest.sobrerrestringido, FlujoHorarioTest.rupturaParcial
RD-10	Mismo docente por grupo	RF-06	Nueva en el M3	restricciones/ RestriccionDocenteUnicoGrupoRD10.java	RestriccionesDurasTest.docenteUnico
RV-01	Tope de 44 con techo	RF-04, RF-05, RF-08	ReglaTopeMatriculadosRV01 y ValidadorEntrada	ConfiguracionParametros.calcularUmbralMaximoGrupo	ReglasValidacionTest.topeMatriculados, ConfiguracionParametrosTest.umbralConTecho, ParametrosApiTest.tope35
RV-02	Cierre con menos de 10	RF-04, RF-05	ReglaCierreMinimoRV02 y ValidadorEntrada	validacion/ ReglaCierreMinimoRV02.java	ReglasValidacionTest.cierreMinimo
Oper.	Carga de inventarios	RF-01 a RF-05	CargadorInventariosCSV	CargadorInventariosCSV, ControladorInventarios y ServicioInventarios	CargadorInventariosCSVTest.demoSinRechazos, FlujoHorarioTest.rupturaCarga
Oper.	Parámetros del semestre	RF-08	ConfiguracionParametros	ControladorParametros y ServicioParametros	ParametrosApiTest.cierre20e invalido
Oper.	Exportar el horario	RF-09	ExportadorHorarioCSV	ExportadorHorarioCSV y GET /api/horarios/{id}/exportar	MotorAsignacionTest.exportacion, FlujoHorarioTest.flujoCompleto
Oper.	Acceso restringido	RNF-06	Seguridad de Spring Boot	ConfiguracionSeguridad, FiltroJwt y ServicioJwt	SeguridadTest.sinToken, credencialesInvalidas, tokenAlterado y tokenValido
Oper.	Ajuste asistido	RF-07	Selección asistida	ServicioHorarios.alternativas y reubicar	FlujoHorarioTest.reubicacionValida
RNF-01	Tiempo de 60 s	RNF-01	MotorAsignacion y EstadoHorario	MotorAsignacion y app.motor.limite-ms	MotorAsignacionTest.demoCompleta, EstresTest
RNF-02	Triple de datos	RNF-02	MotorAsignacion y EstadoHorario	PruebaEstres	EstresTest y docs/evidencias/estres
RF-10	Gestión de catálogos	RF-10	Nuevo en el M3	ServicioCatalogos, ControladorCatalogos y RepositorioCatalogos	CatalogosApiTest.crearEditarBorrarYGenerarConDatosManuales y erroresDeValidacion
RF-11	Cuentas por perfil	RF-11, RNF-06	Nuevo en el M3	ServicioCuentas y ControladorUsuarios	CuentasApiTest.reglasDelUltimoAdminYDeLaSesionActual
RF-12	Publicar el horario	RF-12	Nuevo en el M3	ServicioHorarios.publicar y columna publicado	CuentasApiTest.noSePublicaUnHorarioParcial

ID	Condición del M1	Requisito	Componente M2	Código	Prueba
RF-13	Mover arrastrando	RF-13, RNF-04	Nuevo en el M3	horario.component.ts con Angular CDK y POST /api/horarios/{id}/reubicar	e2e/usuarios.mjs y FlujoHorarioTest.reubicacionValida
RF-14	Portal del docente	RF-14, RNF-06	Nuevo en el M3	ServicioPortal y ControladorPortal	CuentasApiTest.docenteVeSoloSuHorarioPublicadoYCambiaSusDatos
RNF-03	Regla nueva sin tocar el motor	RNF-03	RestriccionDura y lista inyectada	ConfiguracionMotor y RestriccionMismoDiaRD09	ModificabilidadTest.nuevaRestriccion y cambioDeTope
RNF-05	Uso por un no técnico	RNF-05	Tres pantallas de la interfaz	frontend/src/app/paginas con CargaComponent, ParametrosComponent y HorarioComponent y las carpetas gestion y portal	Recorridos e2e/capturas.mjs y e2e/usuarios.mjs (recorrido.log y recorrido_usuarios.log) y prueba con usuarios

Nota. Oper. = condición operativa que no viene del caso de estudio y se declara para que ningún requisito quede huérfano. RB-01 a RB-04 y RE-01 a RE-04 siguen fuera del MVP por decisión heredada del Módulo 1.

Acta de Cierre y Lecciones Aprendidas

Alcance Realmente Logrado

El sistema carga los cuatro inventarios en CSV y rechaza fila por fila lo que no cumple el formato. Valida los grupos con RV-01 y RV-02 antes de generar. Genera el horario con backtracking respetando RD-01 a RD-08 y RD-10. Cuando no encuentra solución entrega un horario parcial con las causas de cada sesión pendiente por código y bloquea la exportación. Permite reubicar una sesión con alternativas que filtra el motor y bloquea la reubicación que viola una regla. Exporta el horario completo en CSV. Todo esto se hace a través de una API protegida con JWT para el perfil administrador y queda registrado quién generó cada horario.

El sistema también tiene dos perfiles. El administrador gestiona docentes, aulas, materias, grupos y cuentas desde la interfaz, mueve sesiones arrastrándolas en la grilla y publica el horario. El docente consulta su horario publicado, marca su disponibilidad y cambia su contraseña.

La interfaz web cubre el ciclo de generación en tres pantallas, la gestión en cuatro y el portal del docente en tres. El recorrido automático del módulo de usuarios crea un docente, lo habilita arrastrándolo, mueve una sesión arrastrándola, publica el horario y entra como docente. Al final comprueba que el docente recibe 403 si intenta generar

(docs/evidencias/capturas_usuarios/recorrido_usuarios.log). Las capturas del recorrido automático están en el Documento 4. Manual de Usuario (PN2_Grupo5_Doc4_ManualUsuario.pdf). El sistema completo se levanta con docker compose up --build y en la prueba del equipo los tres contenedores quedaron sanos en menos de un minuto (docs/evidencias/sistema/contenedores.txt).

Brecha entre el Diseño y la Implementación

La brecha más grande está en la persistencia. El diseño planteaba entidades JPA y el sistema usa repositorios con JdbcTemplate. El cambio no afecta al dominio pero sí al diagrama de componentes, que se actualizó en el Documento 2. La segunda brecha es el precálculo por franja del conteo inicial. Quedó como decisión pendiente en el Módulo 2 y el contrato del Módulo 3 lo incluía, pero el código final conserva el conteo completo. La tercera es RD-10, una regla que el Módulo 1 no tenía y que la construcción hizo visible. Las tres se explican en la Tabla 1.

La cuarta brecha es de alcance y fue una decisión del equipo. El módulo de usuarios por perfil no estaba en el Módulo 1 ni en el diseño del Módulo 2. Se diseñó y se construyó al final del Módulo 3. Sus pruebas son de integración por la API y un recorrido automático, pero no tiene escenarios de calidad propios ni mediciones como el motor. También cambió la técnica de interacción del Módulo 2 al agregar arrastrar y soltar. Estudiantes y directivos siguen sin perfil en el sistema.

Los pendientes de la Tabla 5 del Módulo 2 quedaron así. El conteo factorizado no se implementó. El orden dinámico de sesiones no se comparó con el orden fijo. La prueba con usuarios de la interfaz construida se realizó y se describe en la sección de usabilidad.

El precálculo por franja quedó definitivamente fuera de esta entrega. Con la Tabla 3 el equipo considera que no hacía falta para el tamaño del MVP, pero sigue siendo la primera mejora si la escala crece.

Limitaciones Conocidas

El motor no tiene verificación hacia adelante ni orden dinámico de sesiones. Una carga nueva de inventarios reemplaza la anterior y borra el historial de ejecuciones. Un token emitido sigue vigente hasta vencer aunque la cuenta se desactive después. Los rechazos de la última carga se guardan en memoria y se pierden al reiniciar el backend. Las reglas RV-01 y RV-02 informan los grupos con problemas pero no los excluyen de la generación. Todas las mediciones usan datos sintéticos del generador del equipo y un horario real puede tener restricciones más cruzadas. El intercambio es solo CSV y JSON quedó fuera. No hay restricciones blandas, de modo que el horario es válido pero no busca ser el mejor según preferencias. Solo existen los perfiles administrador y docente. Estudiantes y directivos no tienen acceso. Si los parámetros cambian después de generar un horario, ese horario ya no se puede consultar ni reubicar y el sistema pide generar uno nuevo con el código GRILLA_CAMBIADA.

Lecciones Aprendidas

La primera lección es que medir antes de decidir evitó errores. El Módulo 2 mostró que el orden ingenuo era más rápido en promedio y aun así se mantuvo la heurística por su peor caso. La segunda es que un diseño con Strategy sí se puede extender sin miedo. RD-10 entró al final del proyecto y no rompió nada. La tercera es que un autodiagnóstico honesto paga. Los tres ajustes que el equipo se puso quedaron cerrados con pruebas. La cuarta es que las afirmaciones del documento deben salir de la evidencia y no al revés. Por eso este informe cita el archivo de cada cifra.

La quinta lección viene de la organización del trabajo. El equipo escribió un acuerdo técnico (CONTRATO_M3.md) con el stack, la API, los formatos y las rutas de las evidencias antes de construir. Eso permitió avanzar en el código y en los documentos al mismo tiempo. El acuerdo también tuvo que actualizarse. JPA cambió a JdbcTemplate, el precálculo no entró y apareció RD-10. Un contrato sirve si se mantiene al día y no si se congela.

Autoevaluación del Equipo

Lo que más facilitó esta entrega fue lo aprendido en Programación I-II, Bases de Datos I-II y en Estructuras de Datos. Permitiendo que cada regla fuera una clase intercambiable y que RD-10 se agregara sin tocar el motor. También permitió escoger mapas de ocupación por franja en lugar de recorrer listas. Bases de Datos fue la base del modelo y de las restricciones UNIQUE que bloquean los choques también en PostgreSQL. Ingeniería de Software dio el vocabulario de atributos de calidad y escenarios que organiza este informe. El tema técnico donde el equipo siente que le falta profundidad es la propagación de restricciones, como la verificación hacia adelante y la consistencia de arcos, que podría reducir el retroceso en casos más difíciles. También falta práctica en seguridad con JWT y en el desarrollo con Angular, que fueron nuevos para varios integrantes en este módulo.

Anexo. Uso de Inteligencia Artificial

El equipo usó Claude, un asistente de inteligencia artificial de Anthropic para mejorar la redacción de este informe, como también para poder guiarse sobre el uso adecuado de Docker.

Referencias

Bass, L., Clements, P. y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Lopez-Cruz, O. (s.f.). *Programación de horarios de clases* [Documento base del reto]. Universidad El Bosque.